

Optimizing Volunteer Scheduling and Emergency Response Using Machine Learning



By: Khushi Choudhary
Guided by: Dr. Robin Donatello
& Dr. Edward Roualdes



Agenda

- Problem & Motivation
- Abstract / Introduction
- Literature Review
- Methodology & Stakeholder Analysis
- Technology Stack
- System Architecture
- Database Design
- Authentication & Security
- Data Pipeline & Feature Engineering
- Machine Learning Model
- Volunteer & Admin Interface
- Demo
- Results
- Limitations
- Future Work
- Conclusion

Problem Context & Motivation

- Wildfires are the fastest-growing disaster type in Northern California
- NVADG coordinates animal evacuation across 3 simultaneous shelter sites
- Previous coordination: phone calls, email chains, manual spreadsheets
- Result: double-booking, gaps in coverage, no real-time awareness
- No mechanism to use past event patterns for planning ahead
- Need: centralized, real-time, low-cost system usable during an active disaster





Abstract & Introduction

- Wildfires require highly coordinated emergency response
- NVADG plays critical role in animal evacuation and sheltering
- Traditional scheduling is static and conflict-prone
- This project: web-based scheduling system + ML for real-time decision support
- Role-based access, shift management, conflict prevention, color-coded visualization
- Supervised Poisson regression model trained on 2024 wildfire deployment data

Literature Review

THEME 1

Emergency coordination & decision support

Spontaneous volunteers emerge in crises but integration with formal systems is limited by communication gaps and unclear roles.

| Whittaker, McLennan & Handmer (2015); Twigg & Mosel (2017)

Persistent barriers: insufficient screening, limited comms infrastructure, no standardized digital tools.

| Lunde et al. (2021) — 5-section, 69-item survey of emergency managers

THEME 2

ML for disaster prediction & response

Random forests, neural networks, and SVMs can support event forecasting, resource allocation, and real-time analysis.

| Chamola et al. (2021) — survey of ML in disaster & pandemic management

Wildfire ML spans occurrence, behavior, detection, and management; weather, topography, and fuel are key predictors relevant to staffing demand.

| Jain et al. (2021) — ML in wildfire science

THEME 3

Forecast-to-decision frameworks

Deep learning + stochastic optimization improves timeliness and fairness of pre-shortage allocation vs. reactive approaches.

| Zhang, Zhao, Fang & Chen (2023) — validated on real and simulated disaster data

Existing platforms (e.g. Better Impact) focus on registration, not real-time scheduling or shift-level forecasting.

| Gap in practitioner tooling — no system integrates forecasting with operational coordination

THEME 4 — THE GAP

No applied solution for volunteer-driven animal evacuation

Most works are either deployed tools without ML, or ML research without deployment. Resource-constrained nonprofits remain underserved.

Manual spreadsheets and email lists are insufficient for hourly fluctuations in wildfire staffing demand.

| Synthesis across Whittaker (2015), Chamola (2021), Zhang (2023)

Research gap: No existing system provides real-time, ML-driven volunteer scheduling for animal evacuation operations



Methodology Overview

- Research design: stakeholder-driven, iterative build-test-validate
- Phase 1: requirements via stakeholder interviews
- Phase 2: technology selection and architecture design
- Phase 3: iterative implementation (volunteer UI → admin UI → ML → color coding)
- Phase 4: testing (unit → integration → UAT → load testing)
- Phase 5: deployment and documentation
- ML ran in parallel: data collection → feature engineering → model training → integration



Stakeholder Analysis & Requirements

User Group 1: Administrators (EOC Staff, VI/BI Personnel)

- NVADG Admin(two roles: admin + volunteer)
- Need: create/manage events, set staffing parameters, view real-time data
- Need: generate EOC reports, access predictive analytics, extend/close events

User Group 2: Volunteers

- Need: view active events, select multi-day/multi-hour shifts
- Need: manage own availability, see visual availability indicators
- Need: no double-booking, fast mobile-friendly interaction

Critical requirements from interviews:

- Rapid shift selection (minimum clicks)
- Visual staffing indicators (color-coded)
- Mobile accessibility
- Conflict prevention
- Real-time session persistence (no re-login between shifts)
- Operational reporting: downloadable signups and unique-volunteer-by-location summaries for EOC



Technology Stack & Justification

Framework: Shiny for Python

Reactive programming model — UI auto-updates when data changes

Unified frontend + backend (no separate REST API needed)

Python ecosystem: pandas, statsmodels, gspread all native

Storage: Google Sheets API (gspread v6.0.0+)

Free within standard quotas — zero cost to NVADG

Staff already use Google accounts and sheets

Offline-accessible during network issues

Built-in versioning and collaborative access

ML: statsmodels v0.14.0+

Poisson GLM with interpretable coefficients

Scikit-learn for evaluation utilities

Supporting

Bootstrap 4.5.2 for responsive layout

python-dotenv for secure credential management

rsconnect for Posit/Shinyapps.io deployment



System Architecture

Model Layer:

Google Sheets API connections
(gsread)

`reactive.Value()` for stateful data
(events, signups, sessions)

`VolunteerStaffingModel`
and
`ShiftAvailabilityCalculator`

`data_repository.py` as abstraction
layer

View Layer:

`admin.py` — admin dashboard UI

`volunteer.py` — volunteer
calendar UI

`login_utils.py` — authentication
interface

Bootstrap 4.5.2 responsive
components

Controller Layer:

`main.py` — authentication routing,
auto-close logic

`@reactive.Effect`,
`@reactive.event()` handlers

Shift submission, conflict
detection, prediction pipeline

Database Design: Google Sheets Schema

Name	AR#	Organization	Role	Phone #
Person 1	1234	NVADG	Admin	xxx-xxx-xxxx
Person 2	2345	Butte County	Volunteer	xxx-xxx-xxxx
Person 3	3456	CVET	Volunteer	xxx-xxx-xxxx

Roles (Admin edit only)

Timestamp	Name	Event	Shift Start	Shift End	Day	Date	Location
	Person 1	Test Fire	7:00	14:00	__day	yyyy-mm-dd	Camelot
	Person 2	Test Fire	14:00	21:00	__day	yyyy-mm-dd	Del Oro
	Person 3	Test Fire	17:00	20:00	__day	yyyy-mm-dd	Hotline

Sign ups (Do not edit)

name	start_date	end_date	camelot_min	camelot_max	del_oro_min	del_oro_max	hotline_min	hotline_max	status
Test Fire	yyyy-mm-dd	yyyy-mm-dd	0	5	0	5	0	5	Active
Test Fire 2	yyyy-mm-dd	yyyy-mm-dd	0	10	0	5	0	10	Closed

Daily Staffing Needs



Authentication & Security

Authentication flow:

1. User enters Name + AR# (Animal Response number)
2. `authenticate_user()` fetches all Roles records
3. Case-insensitive name match + exact AR# verification
4. Returns role → routes to admin or volunteer interface

RBAC matrix (key entries):

- Admins: create events, edit staffing, view all signups, download reports, run predictions
- Volunteers: submit own shifts, delete own shifts, view own schedule

Session persistence (localStorage):

- On login: JavaScript stores Name, AR#, Role in `localStorage`
- On page load: JS reads `localStorage`, sends `rehydrateUserData` event to Shiny server
- Server validates and recreates session — no re-login needed
- On logout:
`localStorage.removeItem('nvadgUserData')`

Credential security:

- `GOOGLE_SHEETS_CREDENTIALS` in env variable (never hardcoded)
- `.env` excluded via `.gitignore`
- Service Account limits API scope to Google Sheets/Drive only
- HTTPS enforced on all connections

Data Pipeline & Feature Engineering

Source data — 4 real 2024 wildfire deployments:

Park Fire: 322 hourly records

Thompson Fire: 230 records

Rocky Fire: 79 records

Apache Fire: 42 records

Total: 673 hourly records from
91 unique volunteers

Date range: June 15 – July 27,
2024

Pipeline stages (`dm.py` → NVADGDataManager):

`import_volunteer_signups()` — fetch from Google
Sheets

`clean_signup_data()` — standardize names, dates,
locations

`parse_shift_times()` — handle HH:MM and decimal
hour formats

`expand_to_hourly_data()` — expand each shift into
one row per person per hour

`add_volunteer_features()` — merge roster
metadata, add time-based features

Save to `nvadg_training_data.csv`

Aggregation for modeling:

`prepare_training_data()`
aggregates hourly rows →
shift-level observations

Groups by: Location × Date × Shift
Category

Target: count of unique volunteers
per group

Features used:

`days_into_event` — temporal
dynamics (Day 0, 1, 2...)

`shift` — 7 time bins: 07-10, 10-12,
12-14, 14-16, 16-18, 18-20, 20-23

`location` — Del Oro, Camelot,
Hotline



Machine Learning Model

```

$$Y_i \sim \text{Poisson}(\lambda_i)$$

$$\log(\lambda_i) = \beta_0 + \beta_1 \cdot \text{days\_into\_event}_i + \beta_2 \cdot \text{shift}_i + \beta_3 \cdot \text{location}_i$$


where  $i = 1, \dots, N \times 3 \times 7$  (each unique location-shift-day combination)


```

Y_i = Observed volunteer count; λ_i = Expected volunteer count

Why Poisson:

- Non-negative integer count outcome
- Log-linear link handles zero-inflation naturally
- Interpretable coefficients — critical for operational trust

Companion daily unique model:

- Daily unique volunteers $\sim$ days_into_event + location
- De-duplicates headcount across shifts
- AIC: 47.44

Implementation (model.py — VolunteerStaffingModel):

- train_model() — fits GLM via statsmodels
- predict() — returns λ per location $\times$ day $\times$ shift
- predict_with_constraints() — applies admin min/max bounds
- predict_full_schedule() — full $3 \times N \times 7$ grid
- $\pm 1.96 \times \sqrt{\lambda}$

Auto-retrain:

- TTriggered on event close. New signups appended to training data, model re-fit and saved. Falls back to % of max if model unavailable.



Demo

<https://datascienceatchico-nvadg.share.connect.posit.cloud/>

Username: Full Name

Password: First 4 letters of your Chico State Email





Admin Interface

- Create New Event
- Manage Events
- View Signups
- Admin Calendar
- ML Staffing Predictions

Volunteer Interface

- Sidebar details
- Calendar Features
- Conflict detection
- Continuous Time Consolidation
- Behaviours
- Session Behaviour



Results:

- Volunteer login, event selection, multi-day shift submission ✓
- Continuous time block consolidation (7am-4hr selection → single 7:00-11:00 entry) ✓
- Conflict detection — rejection of overlapping hours ✓
- Real-time calendar color-coding using ML predictions ✓
- Admin event create/edit/extend/close ✓
- Automatic event closure on session startup ✓
- CSV report download with correct volunteer counts ✓
- AI staffing prediction grid with uncertainty ranges ✓
- Auto-retrain on event close ✓
- Session persistence across page refresh ✓
- Deployment done on PositCloud ✓



Limitations

DATA

Small, imbalanced training set

- **673 records** across 4 events, June–July 2024 only — limited temporal range
- **Severe location imbalance:** Del Oro (612), Camelot (61), Hotline (0) — Hotline entirely absent from training
- Only **30 shift-level aggregated observations** used for model fitting

MODEL

Scope of predictors and metrics

- Features limited to **days into event, shift, and location** — no animal intake, workload, or fire severity data
- Fire behavior (severity, spread, evacuation dynamics) is **not modeled** — forecasts reflect historical volunteer patterns only
- Sparse shifts make **MAPE misleading** — MAE and RMSE are the primary error metrics

SYSTEM

Functional gaps for future work

- Admins **cannot directly assign** volunteers to shifts — self-scheduling only
- **No waitlist** when a shift reaches max capacity
- **No push notifications** — volunteers must check availability manually
- **No built-in analytics dashboard** for operational reporting



Future Work

Communication & engagement

- Notifications (email/SMS) for new events and reminders; configurable session timeout
- Capacity warnings as locations approach max; stronger mobile layout

Scheduling & staffing

- Admin-assigned shifts; waitlists; volunteer preferences and skill-aware matching

Reporting & platform

- Built-in analytics, staffing-gap views, and exports (e.g. PDF/Excel)
- PostgreSQL (or similar) if concurrent use and data volume outgrow Sheets

Integrations

- Better Impact / VI-BI alignment; calendar (e.g. iCal) and API access for other tools

Model & data

- Real animal intake and fire-severity signals; more balanced historical events by location
- Clearer uncertainty in the UI (beyond current bands)



Conclusion

- Built: end-to-end emergency scheduling platform — deployed, role-based, real-time
- Integrated: interpretable Poisson regression into live calendar UI
- Validated: stakeholder interviews, unit tests, integration tests, UAT, cross-browser, load testing
- Contribution: practical blueprint for ML-integrated, zero-infrastructure nonprofit coordination tools
- Core insight: emergency coordination improves when prediction and scheduling are unified in one interface



Thank You!